

(192572099) CO1 → AT1 Short Answer Analysis

Suppose a modern OS supports Multiple users and Processes with shared resources like CPU, Memory and I/O devices.

(a) Classify different types of OS Suitable for this scenario & justify your answer

(b) Draw the structure of an OS showing how resources are managed

(c) Write a sequence of system calls required to create & execute a new process in such an environment.

Q2) Consider an OS Managing multiple files & devices for concurrent users.

(a) Categorize system calls used for file & device Management

(b) Illustrate how the OS handle file operations Using a layered structure diagram

(c) Write a pseudo-code (or) C-based system call sequence to open, read & close a file.

Q3) Assume a real-time OS Used in an embedded system controlling industrial Machinery

(a) Identify the type of OS & explain why it is suitable for real-time applications

(b) Draw a diagram representing the interaction b/w Hardware, kernel & user processes.

(c) List & explain system calls required for process synchronization in this system

Q4) A distributed Computing system consists of multiple Interconnected Computers sharing resources over a Network

- a) Classify the type of OS used & compare it with centralized OS
- b) Draw the architecture of a distributed OS showing Communication b/w nodes
- c) Write the sequence of system calls involved in Interprocess Communication.

Q5) Consider a scenario where an OS must manage processes Scheduling, Memory allocations & device Communication efficiently

- a) Explain the Major functions of an OS in handling these tasks
- b) Draw a block diagram representing OS Components & their interactions
- c) Write system call examples for memory allocation and device Management

(a) Types of Operating System

For Multiple users and Multiple process, the suitable OS types are

→ Multi-User Operating System

⇒ It allows many users to use the Computer Simultaneously

→ Multitasking Operating System

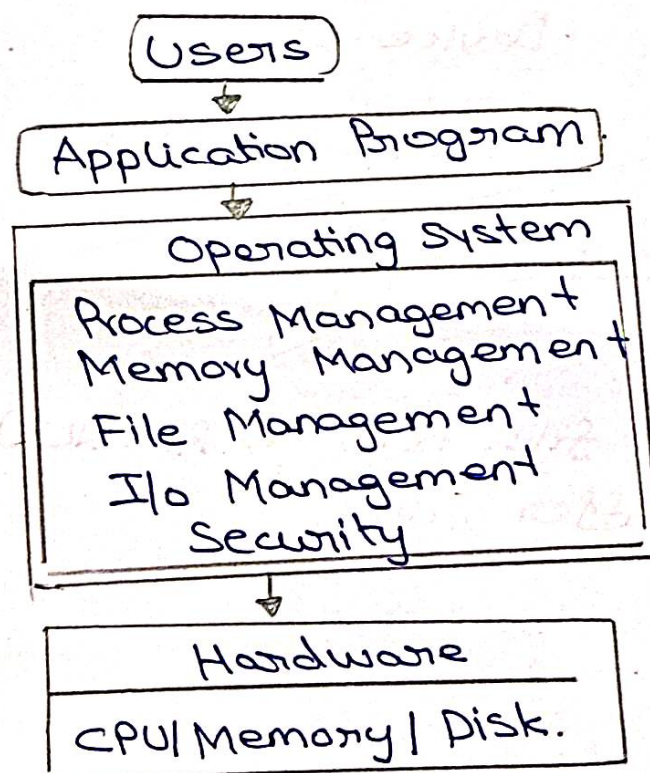
⇒ It runs Multiple programs at the Same time

→ Time sharing Operating System:

⇒ It shares CPU timely fairly among Users

These OS types are efficiently share CPU, Memory files, I/O devices among Multiple users and Processes.

(b) Operating System Structure



(c) System Call Sequences

fork() → Create a new process

exec() → Load and execute new program

wait() → Parent wait for child

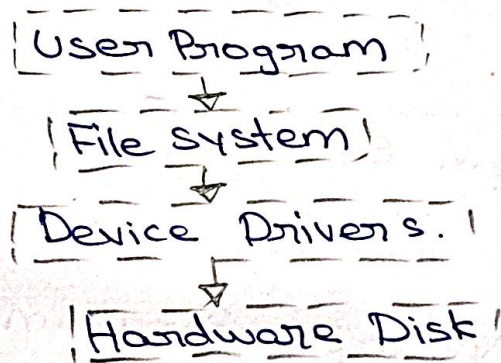
exit() → child terminates

Q2)

(a) File and Device Management System Calls.

<u>File Management</u>	<u>Device Management</u>
open()	request device
read()	read device
write()	write device
close()	release device
delete()	

(b) Layered Structure



(c) C System Call Example

```
int fd;
```

```
fd = open("file.txt", O_RDONLY);
```

```
read(fd, buffer, 100);
```

```
close(fd);
```


System Call Sequence

open()
↓
read()
↓
close()

3)

(a) Types of Operating System?

RTOS

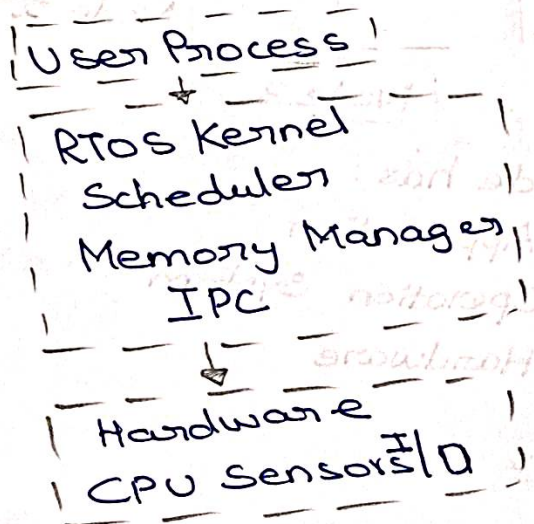
Real Time Operating

System.

Because;

- Fast response
- Predictable execution
- Meets strict deadlines
- High Reliability
- Used in Industrial Machines and Robotics

(b) RTOS Structure



(c) Synchronization System Calls

- Semaphore - wait()
- Semaphore - signal()
- Mutex lock()
- Mutex - unlock()

These synchronize processes and prevent race condition

Q4)

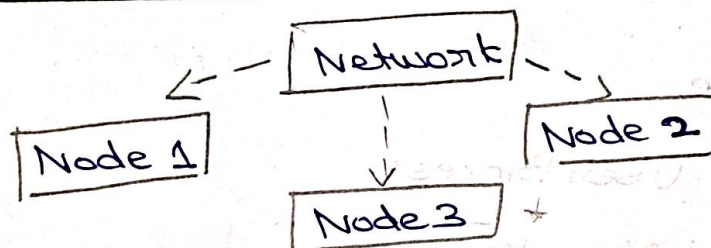
(a) Type of OS

Distributed operating system

Comparison

Distributed OS	Centralized OS
Many Computers	One Computer
Resource Sharing	Local Resources
High Reliability	Less Reliable
Better Scalability	Limited Scalability

(b) Distributed OS Architecture



Each node has :

- Application
- Operation System
- Hardware

(c) IPC System Calls

pipe()
socket()
send()
receive()
close

Sequence

pipe() / socket()

↓
send()

↓
receive()

↓
close

Q5) (a) Major function of OS

Process Management:

It creates processes

Schedules CPU

Terminates processes

Memory Management:

It allocates Memory

Free Memory

Protects Memory.

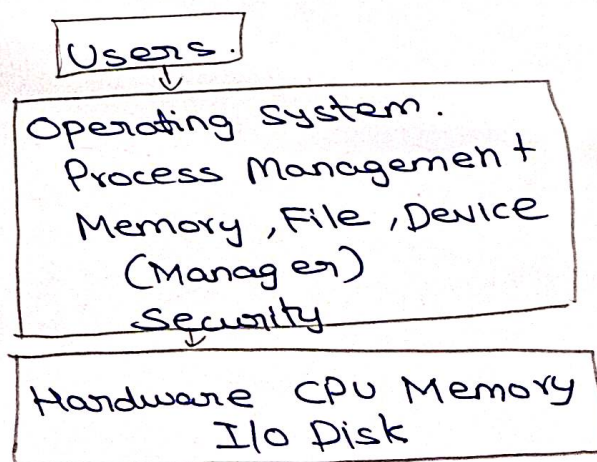
Device Management:

It controls Input/Output devices

Uses device drivers

Allocates device

(b) Block Diagram



(C) System call examples

Memory Allocations

```
ptr = malloc(100);  
free(ptr);
```

Device Management

```
open ("/dev/printer");  
read();  
write();  
close();
```